

Molecular Dynamics Simulation in CCOptimization

A Detailed Code Walkthrough

CCO-projects — `generalized_stealthy` branch

April 28, 2026

Contents

1	Overview	2
2	The Stealthy Potential	2
2.1	Collective-coordinate energy	2
2.2	Constrained wavevector set — generalized stealthy shells	2
2.3	Force on particle i	3
2.4	Soft-core repulsion	3
3	The ParticleMolecularDynamics Class	3
3.1	Member variables	3
3.2	Acceleration computation — <code>_GetAcc</code>	3
4	The Velocity-Verlet Integrator — Evolve	3
4.1	Algorithm (one step)	4
4.2	Code	4
5	The Anderson Thermostat — AndersonEvolve	5
5.1	Purpose	5
5.2	Mechanism	5
5.3	Box-Muller transform	5
5.4	Kick rate	5
6	Velocity Initialization — SetRandomSpeed	6
7	The Simulation Pipeline — CollectiveCoordinateMD	6
7.1	Phase 0: Initial quench	6
7.2	Phase 1: Equilibration (Stage 1)	6
7.3	Phase 2: Sampling (Stage 2)	7
7.4	Timeline summary	7
8	Checkpoint and Restart — <code>restore</code>	7

9 Stopping Criterion	8
10 The SLURM Script — <code>ex_MD_slurm.sh</code>	8
10.1 Parameter encoding	8
10.2 Shell string construction	8
10.3 CLI call	9
10.4 Key parameters	9
11 Data Flow Diagram	9
12 Summary	10

1 Overview

The `CCOptimization` module implements NVT molecular dynamics (MD) simulations of point particles interacting through a *stealthy collective-coordinate potential*, optionally combined with a soft-core repulsion. The goal is to sample finite-temperature configurations in which the structure factor $S(\mathbf{k})$ is suppressed (or exactly zero at $T = 0$) over a prescribed union of annular shells in reciprocal space.

The MD pipeline spans three files:

- `Potential/MD_System.h / MD_System.cpp` — the integrator and thermostat (`ParticleMolecularDynamics` class).
- `CCOptimization/main.cpp` — the CLI driver; reads parameters, builds the potential, and orchestrates the simulation stages.
- `CCOptimization/ex_MD_slurm.sh` — the SLURM submission script; sets physical and computational parameters and calls the executable.

2 The Stealthy Potential

2.1 Collective-coordinate energy

The interaction energy is the collective-coordinate (CC) potential

$$\Phi = \frac{1}{V} \sum_{\mathbf{k} \in \mathcal{C}} \tilde{V}(\mathbf{k}) |\hat{\rho}(\mathbf{k})|^2 \quad (1)$$

where V is the volume of the periodic simulation cell, $\mathcal{C}$ is the set of constrained wavevectors, and

$$\hat{\rho}(\mathbf{k}) = \sum_{j=1}^N e^{i\mathbf{k} \cdot \mathbf{r}_j} = \rho_{\text{real}}(\mathbf{k}) + i \rho_{\text{imag}}(\mathbf{k}) \quad (2)$$

is the collective coordinate (structure factor amplitude). The weight function $\tilde{V}(\mathbf{k})$ controls the shape of the potential within the excluded region (flat, overlap, or power-law; set via the `vtilde` argument).

2.2 Constrained wavevector set — generalized stealthy shells

On the `generalized_stealthy` branch, the excluded region is a *union of M annular shells*:

$$\mathcal{C} = \left\{ \mathbf{k} : |\mathbf{k}| \in \bigcup_{n=1}^M [k_1^{(n)}, k_1^{(n)} + \delta^{(n)}] \right\} \quad (3)$$

Each shell n is specified by a lower bound $k_1^{(n)}$ and a width $\delta^{(n)}$, both in unit-number-density units. The special case $M = 1$, $k_1^{(1)} = 0$ recovers the original single-shell stealthy potential.

2.3 Force on particle i

Taking the gradient of Eq. (1) with respect to $\mathbf{r}_i$:

$$\mathbf{F}_i^{\text{CC}} = -\frac{2}{V} \sum_{\mathbf{k} \in \mathcal{C}} \tilde{V}(\mathbf{k}) \left[\rho_{\text{imag}}(\mathbf{k}) \cos(\mathbf{k} \cdot \mathbf{r}_i) - \rho_{\text{real}}(\mathbf{k}) \sin(\mathbf{k} \cdot \mathbf{r}_i) \right] \mathbf{k} \quad (4)$$

2.4 Soft-core repulsion

When `vareps0` > 0, a contact (soft-core) potential is added:

$$\Phi_{\text{rep}} = \varepsilon_0 \sum_{i < j} \left(1 - \frac{r_{ij}}{\sigma_i + \sigma_j} \right)^2 \Theta(\sigma_i + \sigma_j - r_{ij}) \quad (5)$$

where σ_i is the radius of particle i and Θ is the Heaviside step function. The total energy is $\Phi_{\text{tot}} = u_0 \Phi_{\text{CC}} + e_0 \Phi_{\text{rep}}$.

3 The ParticleMolecularDynamics Class

Defined in `Potential/MD_System.h`, this class owns the full dynamical state of the system.

3.1 Member variables

Variable	Type	Description
Position	Configuration	Particle positions (relative coords)
Velocity	<code>vector<GeometryVector></code>	Velocities, one per particle
Acceleration	<code>vector<GeometryVector></code>	Accelerations (= forces / mass)
Mass	<code>vector<double></code>	Particle masses (all set to 1.0)
TimeStep	<code>double</code>	MD time step Δt
Time	<code>double</code>	Cumulative simulation time

3.2 Acceleration computation — `_GetAcc`

```

1 void ParticleMolecularDynamics::_GetAcc(Potential & potential)
2 {
3     potential.AllForce(this->Acceleration);    // F_i for all i at
4     once
5     for(int i=0; i<this->Acceleration.size(); i++)
6         this->Acceleration[i].MultiplyFrom(1.0 / this->Mass[i]);
7 }
```

`AllForce` calls `Force(i)` for every particle i , which evaluates Eq. (4). Crucially, $\rho(\mathbf{k})$ is computed once for the full configuration before any force is evaluated (via the `RhoValid` dirty-bit cache).

4 The Velocity-Verlet Integrator — Evolve

`Evolve(n, potential)` advances the system by n time steps of size Δt using the standard velocity-Verlet algorithm. **All N particles are updated at every step.**

4.1 Algorithm (one step)

$$\mathbf{v}_i(t + \frac{\Delta t}{2}) = \mathbf{v}_i(t) + \frac{\Delta t}{2} \mathbf{a}_i(t) \quad (\text{half-kick, all } i) \quad (6)$$

$$\mathbf{r}_i(t + \Delta t) = \mathbf{r}_i(t) + \Delta t \mathbf{v}_i(t + \frac{\Delta t}{2}) \quad (\text{drift, all } i) \quad (7)$$

$$\mathbf{a}_i(t + \Delta t) = \mathbf{F}_i/m \quad (\text{force eval, all } i \text{ globally}) \quad (8)$$

$$\mathbf{v}_i(t + \Delta t) = \mathbf{v}_i(t + \frac{\Delta t}{2}) + \frac{\Delta t}{2} \mathbf{a}_i(t + \Delta t) \quad (\text{half-kick, all } i) \quad (9)$$

4.2 Code

```

1 void ParticleMolecularDynamics::Evolve(size_t repeat, Potential &
  potential)
2 {
3     for(size_t u=0; u<repeat; u++)
4     {
5         // half-kick
6         for(long i=0; i<this->NumParticle; i++)
7             this->Velocity[i].AddFrom(0.5*this->TimeStep*this->
              Acceleration[i]);
8
9         // drift
10        for(long i=0; i<this->NumParticle; i++){
11            GeometryVector newCart = this->Position.
              GetCartesianCoordinates(i)
12                                     + this->TimeStep*this->
              Velocity[i];
13            this->Position.MoveParticle(i,
14            this->Position.CartesianCoord2RelativeCoord(
              newCart));
15        }
16
17        // force re-evaluation
18        potential.SetConfiguration(this->Position);
19        this->_GetAcc(potential);
20
21        // half-kick
22        for(long i=0; i<this->NumParticle; i++)
23            this->Velocity[i].AddFrom(0.5*this->TimeStep*this->
              Acceleration[i]);
24
25        this->Time += this->TimeStep;
26    }
27 }

```

Velocity Verlet is symplectic and time-reversible, making it suitable for MD. Energy conservation (in the microcanonical NVE sense) depends on the choice of Δt .

5 The Anderson Thermostat — AndersonEvolve

5.1 Purpose

AndersonEvolve implements the **Anderson thermostat** to maintain NVT (constant temperature) dynamics. It augments each velocity-Verlet step with stochastic velocity resets that couple the system to a fictitious heat bath at temperature T .

5.2 Mechanism

After each complete velocity-Verlet step, a running accumulator `ResetCount` is incremented by `ResetSpeedProbability` ($= 0.01$). Whenever `ResetCount` ≥ 1 , one randomly chosen particle has its velocity completely redrawn from the Maxwell-Boltzmann distribution at temperature T :

```

1 ResetCount += ResetSpeedProbability;          // 0.01 per step
2 for (; ResetCount > 0; ResetCount -= 1.0)
3 {
4     size_t i = floor(gen.RandomDouble() * NumParticle); //
5     // random particle
6     double sigma = sqrt(kT / Mass[i]);
7     for (DimensionType j = 0; j < Dimension; j++)
8     {
9         // Box-Muller: Gaussian with std = sigma
10        this->Velocity[i].x[j] =
11            sqrt(-2*log(gen.RandomDouble()))
12            * sin(2*pi*gen.RandomDouble())
13            * sigma;
14    }
15 }
```

5.3 Box-Muller transform

The new velocity component is drawn from a Gaussian via the Box-Muller transform. If $u_1, u_2 \sim \mathcal{U}(0, 1)$, then

$$v_j = \sqrt{-2 \ln u_1} \sin(2\pi u_2) \sigma, \quad \sigma = \sqrt{k_B T / m} \quad (10)$$

is Gaussian with mean 0 and variance $k_B T / m$, which is the Maxwell-Boltzmann distribution for a single velocity component:

$$P(v_j) \propto \exp\left(-\frac{mv_j^2}{2k_B T}\right) \quad (11)$$

5.4 Kick rate

With `ResetSpeedProbability` $= 0.01$:

- One velocity reset occurs every **100 steps** (deterministic, via the accumulator).
- The chosen particle is **random** each time.

- For $N = 100$ particles, each particle is reset on average once every $100 \times 100 = 10,000$ steps.

The thermostat acts weakly: most dynamics between kicks are purely Newtonian, preserving realistic trajectories while maintaining the target temperature over long times.

6 Velocity Initialization — SetRandomSpeed

```

1 void ParticleMolecularDynamics::SetRandomSpeed(double kT,
2   RandomGenerator & gen)
3 {
4     for(size_t i=0; i<this->NumParticle; i++)
5     {
6         double sigma = sqrt(kT / this->Mass[i]);
7         for (DimensionType j = 0; j < this->Dimension; j++)
8             this->Velocity[i].x[j] =
9                 sqrt(-2*log(gen.RandomDouble()))
10                * sin(2*pi*gen.RandomDouble()) * sigma;
11     }
12 }
```

This is an *instantaneous* reset of **all** $N \times d$ velocity components simultaneously, drawn from the Maxwell-Boltzmann distribution at temperature T . It is used during equilibration (Stage 1) to aggressively drive the system from $T = 0$ to the target temperature.

7 The Simulation Pipeline — CollectiveCoordinateMD

The full MD run is managed by CollectiveCoordinateMD in CCOptimization/main.cpp. It proceeds through three phases.

7.1 Phase 0: Initial quench

```

1 RelaxStructure_NLOPT(*pConfig, *pPotential, 0.0, 0, 0.0, 1000);
2 psystem = new ParticleMolecularDynamics(*pConfig, TimeStep, 1.0);
```

The initial configuration (random or loaded) is first minimized to a local energy minimum via LBFGS (RelaxStructure_NLOPT). This ensures the MD starts from a ground state at $T = 0$ rather than a high-energy random configuration. The ParticleMolecularDynamics object is then created with all velocities set to zero.

7.2 Phase 1: Equilibration (Stage 1)

Runs EquilibrateSamples blocks (set by numequilibration). Each block:

1. SetRandomSpeed(T) — redraw *all* velocities from Maxwell-Boltzmann at T . This is a complete velocity reset, not the Anderson thermostat.
2. Evolve(StepPerSample/2) — pure NVE for $n/2$ steps.
3. Repeat steps 1–2 once more.

Total steps in equilibration: `numequilibration` $\times$ `samplesteps`. **No configurations are saved.**

The purpose is to heat the system rapidly from $T = 0$ to the target temperature. The brute-force velocity resets are intentionally aggressive — they are not physically realistic but quickly establish the correct kinetic energy scale.

7.3 Phase 2: Sampling (Stage 2)

Runs `SampleNumber` = `Nc` blocks. Each block:

1. `AndersonEvolve(StepPerSample/2)` — velocity Verlet with Anderson thermostat for $n/2$ steps.
2. `Evolve(StepPerSample/2)` — pure NVE for $n/2$ steps.
3. **Save** the current configuration to the output `ConfigPack`.

One configuration is saved per block, separated by `samplesteps` $\times$ Δt time units.

7.4 Timeline summary

Phase	Blocks	Steps/block	Saved?
Quench (LBFGS)	1	up to 1000 evals	No
Equilibration	<code>numequilibration</code>	<code>samplesteps</code>	No
Sampling	<code>Nc</code>	<code>samplesteps</code>	Yes (1 per block)

With the example script parameters (`numequilibration`=10, `samplesteps`=1000, `Nc`=20, Δt =0.05):

Phase	Total steps	Total time units
Equilibration	10,000	500
Sampling	20,000	1000
Total	30,000	1500

8 Checkpoint and Restart — restore

When `restore` is passed, the code saves a binary checkpoint `fname_MD.MDDump` at the end (or when the wall-clock limit is reached). The checkpoint stores the full dynamical state: position, velocity, acceleration, time, stage counter, and step counter.

On the next invocation (same command with `restore`), the code detects the checkpoint file, loads it, and **continues the trajectory from where it stopped** — no re-equilibration, no loss of progress.

This is essential for long cluster jobs. The wall-clock time limit is checked at the start of every block:


```

1 if (std::time(nullptr) > TimeLimit || std::time(nullptr) > ::
    TimeLimit)
2 {
3     // save .MDDump checkpoint
4     // write fname_continue.txt warning
5     return 0;
6 }

```

9 Stopping Criterion

The code has **no automatic steady-state detection**. It stops when:

1. All `Nc` sampling blocks are completed, or
2. The wall-clock time limit (`timelimit` hours) is reached.

Whether the system has truly equilibrated is the user's responsibility. The log prints E_p (potential energy) and E_k (kinetic energy) at each saved snapshot:

```

1 std::cout << "2:" << i << "/" << SampleNumber
2           << " \tE_relax=" << Ep
3           << " \tE_k=" << Ek << '\n';

```

The user should inspect these values and verify that E_p has stabilized before using the saved configurations for analysis.

10 The SLURM Script — ex_MD_slurm.sh

10.1 Parameter encoding

Physical k-space parameters are specified at unit number density and converted to absolute units via:

$$a = \left(\frac{\phi_{\text{fake}}}{V_d} \right)^{1/d}, \quad K_{1,a}^{(n)} = k_1^{(n)} \cdot a, \quad \delta_a^{(n)} = \delta^{(n)} \cdot a \quad (12)$$

where V_d is the volume of a unit hypersphere in d dimensions. The code divides back by a to recover absolute values. This encoding makes the script parameters independent of N and box size.

10.2 Shell string construction

```

1 shell_str="${M}"
2 for (( n=0; n<M; n++ )); do
3     K1a='echo "${K1s[$n]}*${a}" | bc -l'
4     deltaa='echo "${deltas[$n]}*${a}" | bc -l'
5     shell_str="${shell_str} ${K1a} ${deltaa}"
6 done

```

10.3 CLI call

```

1 ${exe} ${timelimit} ${seed} ${beg_idx} ${Verbosity} <<< \
2   "${d} ${shell_str} $S0 $vareps0 ${sigma} ${phi_fake} \
3   ${threads} ${N} ${Nc} random ${fname}_MD \
4   MD $T $dt $int_samp $num_equi $restore run"

```

10.4 Key parameters

Script variable	Default	Description
<code>d</code>	2	Spatial dimension
<code>M</code>	2	Number of stealthy shells
<code>K1s</code>	(0.0, 0.5)	Lower bounds per shell (unit density)
<code>deltas</code>	(0.3, 0.2)	Widths per shell (unit density)
<code>S0</code>	0.0	Target $S(\mathbf{k})$; 0 = stealthy
<code>vareps0</code>	1.0	Soft-core repulsion strength
<code>N</code>	100	Number of particles
<code>Nc</code>	20	Configurations to collect
<code>temperature</code>	10^{-3}	Dimensionless temperature (units of v_0)
<code>timestep</code>	0.05	MD time step Δt
<code>samplesteps</code>	1000	Steps between saved snapshots
<code>numequilibration</code>	10	Equilibration blocks
<code>timelimit</code>	4	Wall-clock limit (hours)

11 Data Flow Diagram

Step	Action
1	<code>ex_MD_slurm.sh</code> : encode k -space shells, build CLI string
2	<code>main.cpp</code> / <code>GetCCO</code> : parse stdin, build constraint list $\mathcal{C}$
3	<code>RepulsiveCCPotential</code> : store $(\mathbf{k}, \tilde{V}(\mathbf{k}))$ for each $\mathbf{k} \in \mathcal{C}$
4	<code>CollectiveCoordinateMD</code> : initial LBFGS quench
5	Stage 1: <code>SetRandomSpeed</code> + <code>Evolve</code> $\times$ <code>numequilibration</code>
6	Stage 2: <code>AndersonEvolve</code> + <code>Evolve</code> $\times$ <code>Nc</code> , save each
7	Output: <code>fname_MD.ConfigPack</code> with <code>Nc</code> configurations

12 Summary

- The integrator is **velocity Verlet** — symplectic, time-reversible, applied to *all* N particles at every step.
- The thermostat is **Anderson**: one randomly chosen particle has its velocity redrawn from Maxwell-Boltzmann every 100 steps. Between kicks, dynamics are purely Newtonian (NVE).
- **Equilibration** uses brute-force full velocity resets (`SetRandomSpeed`) to heat the system from $T = 0$ rapidly. No configs are saved.
- **Sampling** switches to the gentle Anderson thermostat. One configuration is saved every `samplesteps` $\times \Delta t$ time units.
- There is **no automatic equilibration check**. The user must inspect E_p convergence in the log.
- **Checkpointing** via `restore` allows seamless continuation across SLURM job submissions.
- The `generalized_stealthy` branch supports M annular shells in k -space; the script builds the shell string automatically.